

REPORT ON CRYPTOGRAPHY

1. INTRODUCTION

Cryptography is the science of protecting information by transforming readable data into an unreadable form. It plays a major role in securing communication systems, banking systems, military communication, and internet applications. The primary objectives of cryptography are confidentiality, integrity, authentication, and non-repudiation.

Practical Uses of Cryptography

Cryptography is used to secure communication and protect information from unauthorized access. It ensures confidentiality, integrity, authentication, and non-repudiation of data in various real-world applications.

1. Secure Communication

Cryptography is used to protect messages and data transmitted over networks such as the Internet. Applications like email services, messaging apps, and video conferencing platforms use encryption to prevent unauthorized users from reading the information.

2. Online Banking and Digital Payments

Banks and financial institutions use cryptographic techniques to secure online transactions, ATM operations, credit/debit card payments, and mobile banking services. It helps in protecting sensitive financial information from fraud and cyber attacks.

3. Password Protection

Passwords are stored in encrypted or hashed form using cryptographic algorithms. This prevents attackers from directly accessing user credentials even if the database is compromised.

4. Data Integrity Verification

Cryptography ensures that data is not altered during transmission or storage. Hash functions are used to verify the integrity of files, software, and digital documents.

5. Authentication and Digital Signatures

Digital signatures and authentication mechanisms use cryptography to verify the identity of users and confirm the authenticity of electronic documents and transactions.

6. Secure Web Browsing

Protocols such as HTTPS use cryptography to establish secure connections between web browsers and websites, protecting user information from interception.

7. Military and Government Applications

Governments and defense organizations use cryptography to secure confidential communications, classified documents, and national security information.

8. Cloud and Database Security

Cryptography is used to encrypt stored data in cloud systems and databases, ensuring that only authorized users can access sensitive information.

9. E-commerce Applications

Online shopping platforms use cryptographic techniques to secure customer information, payment details, and transaction records.

10. Blockchain and Cryptocurrencies

Cryptography forms the foundation of blockchain technology and cryptocurrencies by securing transactions and maintaining trust in decentralized systems.

CIA Triad : The **CIA Triad** is the foundation of information security. It represents the three main security goals used to protect data and system

Confidentiality: Confidentiality means protecting information from unauthorized access. Only authorized users should be able to view the data.

Integrity: Integrity means maintaining the accuracy and correctness of data. Data should not be modified by unauthorized users.

Availability: Availability means ensuring that systems and data are accessible whenever needed. Authorized users should get timely access to resources.

CRYPTOLOGY

Cryptology is the study of secure communication techniques. It consists of two branches: Cryptography and Cryptanalysis. Cryptology focuses on developing and analyzing secure systems for communication and data protection.

CRYPTOGRAPHY

Cryptography is the process of converting plaintext into ciphertext using encryption algorithms and keys.

Plaintext: Original readable message

Ciphertext: Encrypted unreadable message

Encryption: Process of converting plaintext into ciphertext

Decryption: Process of converting ciphertext back into plaintext

CRYPTANALYSIS

Cryptanalysis is the study of breaking encryption systems without knowing the secret key.

Types of attacks include: Ciphertext-only attack, Known-plaintext attack, Chosen-plaintext attack, Brute-force attack. Cryptanalysis helps evaluate the strength of encryption algorithms.

SYMMETRIC KEY CRYPTOGRAPHY

In symmetric key cryptography, the same key is used for both encryption and decryption.

Advantages: Fast execution Efficient for large data **Disadvantages:** Key distribution problem Less secure over public networks Examples include DES and AES.

ASYMMETRIC KEY CRYPTOGRAPHY

Asymmetric cryptography uses two keys: Public Key Private Key The public key is used for encryption while the private key is used for decryption.

Advantages: Secure communication Supports digital signatures **Disadvantages:** Slower execution High computational overhead RSA is one of the most widely used asymmetric algorithms.

DES (DATA ENCRYPTION STANDARD)

□ **Definition:** DES is a symmetric block cipher adopted in 1977 by the National Bureau of Standards (now NIST).

□ **Algorithm Name:** It is formally known as the **Data Encryption Algorithm (DEA)**.

□ **Status:** While it was the most widely used encryption scheme for decades, NIST now recommends it only for **legacy systems**, favoring Triple DES (3DES) or AES for modern applications.

2. Basic Parameters

- **Block Size:** 64 bits (data is processed in 64-bit chunks).
- **Key Size:** 56 bits. Although the input key is 64 bits, every 8th bit is ignored (often used for parity), leaving an effective security strength of 56 bits.
- **Structure:** It follows the **Feistel Cipher** structure, consisting of 16 rounds of processing.

3. The Feistel Structure & Process

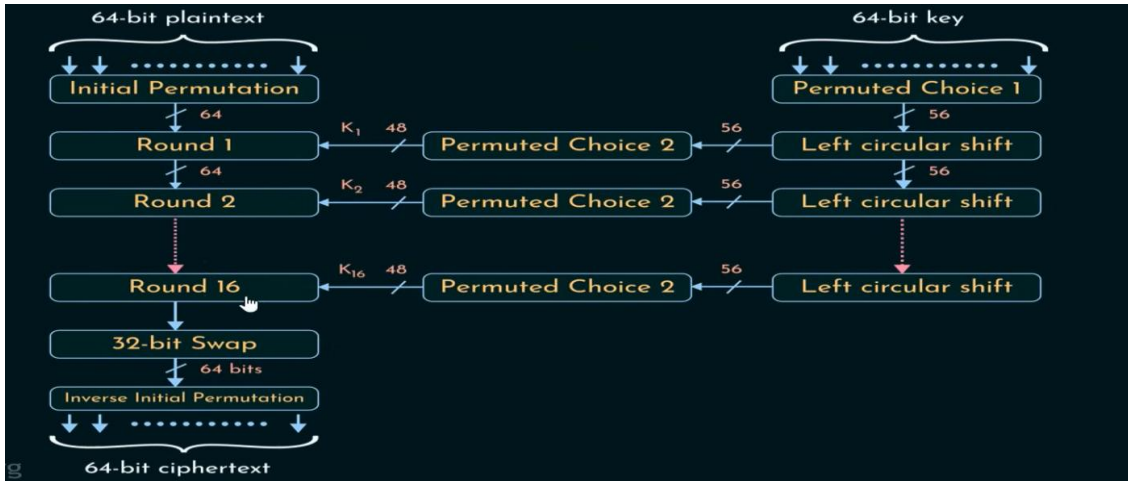
DES is an implementation of a Feistel network, which approximates an "ideal block cipher" by alternating substitution and permutation.

- **Initial Permutation (IP):** The 64-bit plaintext is first rearranged according to a fixed table.

- **16 Rounds:** Each round performs the same set of functions on the data, using a different 48-bit subkey (derived from the main 56-bit key).
 1. **Split:** The 64-bit block is divided into Left (\$L\$) and Right (\$R\$) 32-bit halves.
 2. **Expansion:** The right half (\$R\$) is expanded from 32 to 48 bits to match the subkey length.
 3. **Substitution (S-boxes):** The XORed result of the expanded \$R\$ and the subkey is passed through eight **S-boxes**. These are non-linear substitution tables that provide the core security of DES.
 4. **Permutation (P):** The S-box output is rearranged.
 5. **Swap:** The new \$L\$ becomes the old \$R\$, and the new \$R\$ is the old \$L\$ XORed with the function output.
- **Final Permutation:** After 16 rounds, the left and right halves are swapped and passed through the inverse of the initial permutation (IP^{-1}) to produce the final ciphertext.

4. Security and Key Strength

- **The 56-Bit Controversy:** From its inception, critics argued the 56-bit key was too short. Today, DES is vulnerable to **brute-force attacks**; a system processing 1 million keys per microsecond can crack a DES key in roughly 10 hours.
- **S-Box Secrecy:** The design criteria for the internal S-boxes were originally classified. However, research into **differential cryptanalysis** later showed that the S-boxes were actually designed with high resistance to specific mathematical attacks.
- **Avalanche Effect:** DES exhibits a strong avalanche effect, where a small change in plaintext or key results in a significant, unpredictable change in the ciphertext.



AES (ADVANCED ENCRYPTION STANDARD)

- **Purpose:** AES was published by the National Institute of Standards and Technology (NIST) in 2001 to replace the aging Data Encryption Standard (DES).
- **Type:** It is a **symmetric block cipher**, meaning it uses the same key for both encryption and decryption.
- **Standard Designation:** It is designated as Federal Information Processing Standard 197 (FIPS PUB 197).

AES is highly flexible and supports three different key lengths, though it always uses a fixed block size of 128 bits

Key Size (bits)	Number of Rounds (N)	Block Size (bits)
128 (AES-128)	10	128
192 (AES-192)	12	128
256 (AES-256)	14	128

3. Data Structures

- **State Array:** AES operations are performed on a square matrix of bytes called the **State**. For encryption, the 128-bit plaintext is copied into this 4x4 matrix, modified at each stage, and then copied to the output matrix.

- **State Ordering:** Input bytes are ordered within the matrix **by column** (e.g., the first four bytes occupy the first column).

4. Detailed Structure & Rounds

Unlike DES, AES does **not** use a Feistel structure. Instead, it processes the entire data block as a single matrix using substitution and permutation in every round.

Round Functions : Each full round (except the first and last) consists of four distinct transformation functions:

1. **SubBytes (Substitute Bytes):** A non-linear substitution using a pre-defined S-box to perform byte-by-byte replacement.
2. **ShiftRows:** A simple permutation where rows of the state are shifted circularly by different offsets.
3. **MixColumns:** A transformation that treats each column of the state as a polynomial over $GF(2^8)$ and multiplies it by a fixed polynomial.
4. **AddRoundKey:** A bitwise XOR operation between the current State and a portion of the expanded key (round key).

Execution Sequence

- **Initial Step:** Begins with an AddRoundKey stage (considered Round 0).
- **Intermediate Rounds:** Rounds 1 through $(N-1)$ include all four transformations.
- **Final Round:** The last round (N) omits the MixColumns stage to make the cipher reversible for decryption.

5. Security Characteristics

- **Avalanche Effect:** A small change in either the plaintext or the key produces a significant change in the ciphertext, a critical security property .
- **Non-linearity:** The SubBytes S-box is designed to have no fixed points and a low correlation between input and output bits to resist cryptanalysis.

- **Key Expansion:** The 16, 24, or 32-byte input key is expanded into a much larger key schedule (e.g., 44 words for AES-128) using a specialized algorithm.

6. Implementation Notes

- **Performance:** AES can be implemented efficiently on both 8-bit and 32-bit processors.
- **Decryption:** The decryption process uses the inverse of each transformation function (InvSubBytes, InvShiftRows, etc.) and applies round keys in reverse order.

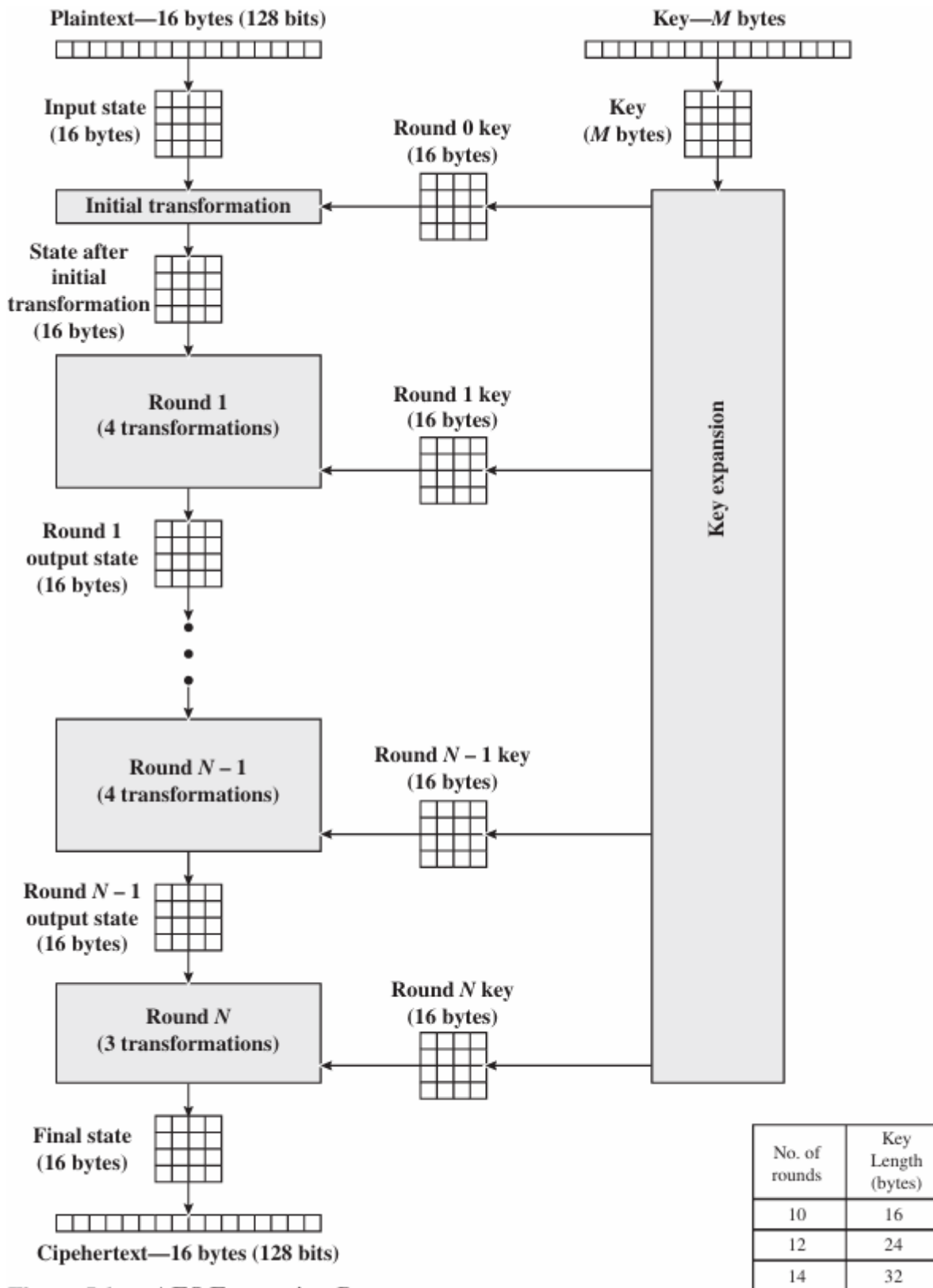


Figure 5.1 AES Encryption Process

1. HTTPS Protocol Overview

Definition: HTTPS (Hypertext Transfer Protocol Secure) is the secure implementation of HTTP. It refers to the combination of standard HTTP and the SSL/TLS protocol to provide secure communication between a web browser and a server. **Core Objectives:**

- **Confidentiality:** Encrypting data to prevent eavesdropping.
- **Integrity:** Ensuring data is not altered or corrupted during transit.
- **Authentication:** Using digital certificates to verify the identity of the website.

2. Cryptography in SSL/TLS

SSL and TLS protocols utilize a hybrid cryptographic approach to balance security and performance:

- **Asymmetric Cryptography (Public-Key):** Used during the initial connection (Handshake) to authenticate the server's identity and securely exchange a session key. Common algorithms include RSA and Diffie-Hellman.
- **Symmetric Cryptography:** Once the handshake is complete, symmetric algorithms (like AES) are used for bulk data encryption. This is much faster and more efficient for transmitting large amounts of data.
- **Message Authentication Codes (MAC):** A keyed hash (like HMAC) is used to ensure that every message sent has not been tampered with by a third party.

3. The TLS Handshake Protocol

The Handshake is the most critical phase, where security parameters are negotiated. It occurs in four distinct phases:

Phase 1: Establish Security Capabilities The client sends a `client_hello` (listing its supported cipher suites and a random number). The server responds with a `server_hello`, choosing the specific encryption algorithms that will be used for the session.

Phase 2: Server Authentication and Key Exchange The server sends its X.509 Digital Certificate, which contains its public key. If the key exchange method requires it (like Diffie-Hellman), the server also sends a `server_key_exchange` message. This phase concludes with a `server_hello_done`.

Phase 3: Client Key Exchange The client verifies the server's certificate. It then generates a "pre-master secret," encrypts it with the server's public key, and sends it back in a `client_key_exchange` message. Both the client and server now use this secret to derive a shared symmetric **Session Key**.

Phase 4: Finalization Both parties send a `change_cipher_spec` message to indicate that all future communication will be encrypted. They then send a finished message containing a hash of the entire handshake to verify that no tampering occurred. After this, application data (HTTP) begins to flow securely.

Cryptographic Hash Functions

A cryptographic hash function H is a fundamental tool that accepts a variable-length block of data M and produces a fixed-size hash value $h = H(M)$, commonly referred to as a **message digest**. For defense applications, its primary objective is ensuring **data integrity**.

- **Core Security Requirements:** To be technically viable, a hash function must satisfy several rigorous criteria:
 - **Preimage Resistance (One-way property):** For any given hash value h , it must be computationally infeasible to find y such that $H(y) = h$.
 - **Second Preimage Resistance (Weak Collision Resistance):** For any given block x , it is infeasible to find $y \neq x$ such that $H(y) = H(x)$.
 - **Collision Resistance (Strong Collision Resistance):** It must be infeasible to find *any* pair (x, y) such that $H(x) = H(y)$.
- **Defense Context:** Hash functions are versatile, used for message authentication, digital signatures, one-way password storage, and identifying modifications in sensitive system files.

Digital Signatures: A **Digital Signature** is a cryptographic mechanism used to verify the **authenticity** and **integrity** of digital data. It acts as a tamper-proof digital seal that binds a sender's identity to a specific document.

The Core Concept : Unlike a handwritten signature, which looks the same on every piece of paper, a digital signature is completely unique to the message it accompanies. It relies on two core cryptographic tools: **Asymmetric (Public-Key) Cryptography** and **Hash Functions**.

How It Works (The Core Mechanism)

Every user has a mathematically linked key pair: a **Private Key** (kept secret by the owner) and a **Public Key** (distributed to everyone). The signing and verification process follows three distinct steps:

1. **Hashing (The Fingerprint):** The sender passes the document through a hash function to create a short, unique mathematical snapshot of the data, known as a digest. If even a single character in the document is altered, this digest changes completely.
2. **Signing (The Lock):** The sender encrypts this digest using their **Private Key**. This encrypted digest is the actual **Digital Signature**, which is then attached to the document and sent.
3. **Verification (The Check):** The recipient decrypts the signature using the sender's **Public Key** to reveal the original digest. At the same time, the recipient calculates a fresh digest of the received document. If both digests match, the signature is valid.

Security Benefits

- **Authenticity:** Because the signature can only be successfully decrypted using the sender's public key, it proves exactly who created it.
- **Data Integrity:** If anyone tampers with the message during transit, the recipient's freshly calculated digest will not match the decrypted signature, instantly exposing the modification.
- **Nonrepudiation:** Since only the sender possesses the private key used to lock the digest, they cannot later claim that they did not send the message.

Blockchain

Blockchain is a decentralized, distributed ledger technology that records data sequentially across a network of computers in a way that is permanently unalterable. It functions as an immutable system where information is stored chronologically without relying on a central authority.

How Data Flows: The Block Lifecycle

Instead of thinking of it as static pieces, the architecture functions as a continuous cryptographic pipeline:

1. **The Transaction/Data Input:** A new activity occurs (such as a file modification or a data transfer) and is broadcast to the peer-to-peer network.
2. **Bundling into a Block:** Independent computers on the network collect these unverified transactions and group them into a structured data container called a **Block**.
3. **Generating the Cryptographic Link:** The block is stamped with a unique mathematical fingerprint using a **Cryptographic Hash Function**. Crucially, this step incorporates the hash of the *previous* block, interlocking them in a permanent timeline.
4. **Achieving Network Consensus:** Before this block can join the ledger, the network nodes must run an automated mathematical protocol (like Proof of Work) to validate the block's data.
5. **Appending to the Chain:** Once validated, the block is permanently appended to the ledger, and every computer updates its copy of the chain simultaneously.

How It Achieves Security

- **Immutability via the Avalanche Effect:** Because each block contains the hash of the previous one, changing even a single bit of data in an old block instantly alters its hash. This causes a mismatch with the hash stored in the next block, breaking the chain and exposing the modification immediately.
- **Distributed Ledger Network:** Instead of storing the notebook on one central server, identical copies of the blockchain are shared across thousands of independent computers (nodes) on the network. For a

hacker to alter the history, they would have to successfully alter the data on a majority of these machines simultaneously, which is computationally infeasible.

- **Consensus Protocols:** The independent computers on the network use automated mathematical algorithms—such as **Proof of Work (PoW)** or **Proof of Stake (PoS)**—to validate and agree on the true state of the ledger before any new block is allowed to be added.

Significance : By shifting trust from a central institution (like a bank or a server administrator) to distributed cryptography, blockchain eliminates single points of failure. This creates a transparent, tamper-proof system for tracking digital assets, logs, and transactions.

Cryptocurrency

A Cryptocurrency is a decentralized digital asset designed to function as a secure medium of exchange. It operates entirely without a central authority, regulatory body, or master server.

How Transactions Flow: The Cryptographic Pipeline

The entire lifecycle of a transaction moves away from centralized authorization and relies instead on an interconnected cryptographic system:

1. **Initiation via Wallets (Key Pairs):** A transaction is initiated using a cryptocurrency wallet, which is built on asymmetric cryptography. The wallet utilizes a public-private key pair: the **Public Key** serves as the transparent account address, and the **Private Key** acts as a secret digital pen.
2. **Authorization via Digital Signature:** To send funds, the sender's system generates a unique **Digital Signature** by encrypting a mathematical snapshot of the transaction order using their **Private Key**. This attached signature provides definitive mathematical proof to the entire network that the legitimate owner authorized the transfer.
3. **Broadcasting & Node Verification:** The signed transaction order is broadcast to a peer-to-peer network of independent computers. Every computer on the network verifies the signature using the sender's **Public Key** to confirm identity, and evaluates the transaction data via **Hash Functions** to verify that it has not been modified in transit.
4. **Consensus and Block Bundling:** Instead of a bank teller checking account balances, the network computers check their local copies of the shared ledger. Valid transactions are collected, grouped into a

data container (a block), and verified by the network through an automated mathematical protocol called a **Consensus Mechanism** (such as Proof of Work or Proof of Stake).

5. **Immutable Ledger Entry:** Once validated by network consensus, the block is hashed, cryptographically linked to the previous block's fingerprint, and permanently appended to the public ledger—the **Blockchain**. The transaction balances are updated across all copies of the ledger simultaneously, making the transfer complete and irreversible.

Smart Contracts: Advanced cryptocurrency platforms expand past basic ledger updates by utilizing **Smart Contracts**. These are self-executing agreements where contractual terms are written directly into lines of code. When predefined mathematical conditions are met, the transaction executes automatically on the decentralized network, eliminating any need for intermediary escrow agents or human administrators.

DES Algorithm

```
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad

msg = input("Enter message: ")

key = b'12345678'

cipher = DES.new(key, DES.MODE_ECB)

encrypted = cipher.encrypt(pad(msg.encode(), DES.block_size))

print("Cipher Text:", encrypted.hex())

decrypted = unpad(cipher.decrypt(encrypted), DES.block_size)

print("Original Text:", decrypted.decode())
```

```
PS C:\Users\tanus\OneDrive\Desktop\DRDO_crypto> python des.py
Enter message: hello
Cipher Text: ba16c6a0257125af
Original Text: hello
```

AES Algorithm

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad

msg = input("Enter message: ")

key = b'1234567890123456'
```

```

cipher = AES.new(key, AES.MODE_ECB)

encrypted = cipher.encrypt(pad(msg.encode(), AES.block_size))

print("Cipher Text:", encrypted.hex())

decrypted = unpad(cipher.decrypt(encrypted), AES.block_size)

print("Original Text:", decrypted.decode())

```

```

PS C:\Users\tanus\OneDrive\Desktop\DRDO_crypto> python aes.py
Enter message: hello
Cipher Text: ebb7c703e675db3da397038b4c17823c
Original Text: hello

```

SHA-256(Hashing)

```

import hashlib

msg = input("Enter message: ")

hash_value = hashlib.sha256(msg.encode())

print("SHA-256 Hash:")
print(hash_value.hexdigest())

```

```

PS C:\Users\tanus\OneDrive\Desktop\DRDO_crypto> python sha.py
Enter message: hello
SHA-256 Hash:
2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824

```

RSA(public key Cryptography)

```

from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP

message = input("Enter message: ").encode()

key = RSA.generate(2048)
public_key = key.publickey()

cipher = PKCS1_OAEP.new(public_key)

encrypted = cipher.encrypt(message)

print("Encrypted:")
print(encrypted.hex())

decipher = PKCS1_OAEP.new(key)
decrypted = decipher.decrypt(encrypted)

print("Decrypted:")
print(decrypted.decode())

```



```

PS C:\Users\tanus\OneDrive\Desktop\DRDO_crypto> python rsa.py
Enter message: hello
Encrypted:
3394289eea758c7a1e727af7b7a1129c938e84fae8269be0d34b4a32ddf269894caefdc605602435be047e073ac52579fbf9c3be7cbab47f11e5b77090
c834e9d586463d7409d6029f266055e686ada1374a8d438aed239c25a6d1d62ebcc075a9e879d9b8770d942cf6ce2505ec4c8c08bd60f4c00e1adcf115
4f0ec55abcc46e9360e0937dd3a281750800a1545c6bc915e2792daa49b34986dfba62bbd1d7a6b8b23a8081c3da1f310a57df5f049ed33f425d6c68d
69e2f1561faa27c50dd815d33861f726fa4da6670a625e3d02fefbfdea06d1c679c8aa706b70bb5c636b33b41821a393f309795d12d9ad0d98767323d8
e9c746cf91d2fe5c98f2f118
Decrypted:
hello

```

Diffie-Hellman

Purpose : Secure key exchange over an insecure channel.

```

p = 23
g = 5

a = 6
b = 15

A = (g ** a) % p
B = (g ** b) % p

key_alice = (B ** a) % p
key_bob = (A ** b) % p

print("Alice Public Key:", A)
print("Bob Public Key:", B)

print("Shared Secret (Alice):", key_alice)
print("Shared Secret (Bob):", key_bob)

```

```

PS C:\Users\tanus\OneDrive\Desktop\DRDO_crypto> python dh.py
Alice Public Key: 8
Bob Public Key: 19
Shared Secret (Alice): 2
Shared Secret (Bob): 2

```